

BDAT 608: Computational Statistics in Big Data II

Model Basics in R

Families, Fitting, Visualisation & Diagnostics

Richard Minkah, PhD and E. Somua-Wiafe, PhD

Department of Statistics & Actuarial Science • University of Ghana, Legon
2025/2026 Academic Year

Introduction to Statistical Models

Fitting a Simple Linear Model

Visualising Models

Formulas and Model Families

Missing Values

Beyond Linear Models

Lecture Outline II

Extended Case Studies

Summary and Exercises

1

Introduction to Statistical Models

What Is a Statistical Model? I

Two essential components:

1. **Family of models** — a generic equation that encodes the *shape* of the relationship.

$$y = a_1 + a_2x \quad (\text{line})$$

$$y = a_1x^{a_2} \quad (\text{power law})$$

2. **Fitted model** — the specific member of the family closest to the observed data.

$$\hat{y} = 4.22 + 2.05x$$

A model provides a **low-dimensional summary** of data by separating *signal* (pattern) from *noise* (residuals).

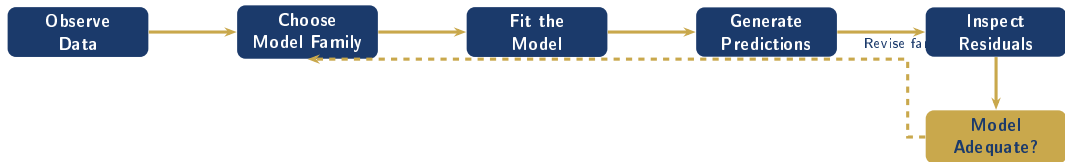
*“All models are wrong,
but some are useful.”*

— George E. P. Box

► Interpretation

The question is never *“Is the model true?”* but *“Is the model useful/adequate in describing the data?”*

Modelling Workflow Overview



► Interpretation

This iterative cycle (**choose, fit, visualise, diagnose**) is the core discipline of applied statistical modelling. We will follow it throughout this section of the course.

Prerequisites & Setup I

```
install.packages(c("tidyverse", "modelr", "splines", "ggfortify", "caret", "mice", "c  
"rpart", "nycflights13"))
```

```
# Load for every session  
library(tidyverse)  
library(modelr)  
library(splines)  
library(dplyr)  
library(ggplot2)  
library(ggfortify)  
library(caret)  
library(mice)    # multiple imputation  
library(cvAUC)  
library(mgcv)
```

Prerequisites & Setup II

```
library(rpart)
library(rpart.plot)
library(nycflights13)

# Ensure R warns when rows are silently dropped
options(na.action = na.warn)

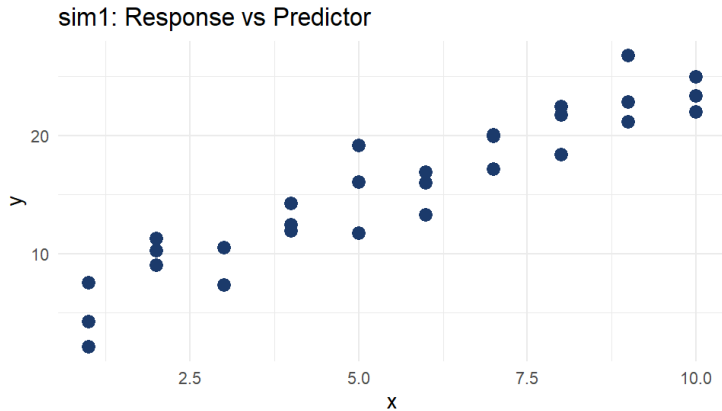
# Explore the sim1 dataset bundled with modelr
glimpse(sim1)
## Rows: 30 Columns: 2
## $ x <int> 1 1 1 2 2 2 3 3 3 ... (1-10, each replicated 3x)
## $ y <dbl> 4.20 7.51 2.13 8.99 ...

# Quick scatter plot
ggplot(sim1, aes(x, y)) +
```



```
geom_point(size = 3, colour = "#1B3A6B") +  
labs(title = "sim1: Response vs Predictor")+  
theme_minimal()
```

Prerequisites & Setup IV



► Interpretation

`modelr` wraps base-R modelling functions so they work naturally in a `dplyr` pipe. `sim1` has 30 rows with a clear linear trend — ideal for building intuition.

2

Fitting a Simple Linear Model

The Linear Model Family I

We assume the relationship between y and x can be described by

$$y = a_1 + a_2 x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2),$$

where a_1 is the **intercept** and a_2 is the **slope**.

What varies across the family?

- Any value of $a_1 \in \mathbb{R}$ (vertical shift)
- Any value of $a_2 \in \mathbb{R}$ (tilt / direction)

What we seek:

The unique pair $(\hat{a}_1, \hat{a}_2)$ that minimises total prediction error.

Randomly generating 250 models

shows the full family. Most are bad: we need a *criterion* to select the best one.

Defining the RMSE Loss Function I

```
model1 <- function(a, data) {  
  a[1] + data$x * a[2]          # intercept + slope * x  
}  
  
# Step 2: Root-Mean-Squared Error (RMSE)  
measure_distance <- function(mod, data) {  
  resid <- data$y - model1(mod, data)  # prediction errors  
  sqrt(mean(resid^2))                  # square root of mean squared error  
}  
  
# Test: RMSE for a candidate model (a1=7, a2=1.5)  
measure_distance(c(7, 1.5), sim1)  
## [1] 2.665212
```

► Interpretation

RMSE = 2.67 means our candidate model misses observed y -values by ≈ 2.67 units on average. The RMSE is **minimised** to find the best (a_1, a_2) . It squares errors before averaging, so large deviations are penalised more severely than small ones.

Alternative Loss: Mean Absolute Error I

```
measure_mae <- function(mod, data) {  
  resid <- data$y - model1(mod, data)  
  mean(abs(resid))                # no squaring -- outliers less influential  
}  
  
# Simulate data with heavy-tailed noise (Student-t, df=2)  
set.seed(42)  
sim1a <- tibble(  
  x = rep(1:10, each = 3),  
  y = x * 1.5 + 6 + rt(length(x), df = 2)    # heavy tails  
)  
  
# Fit using MAE via optim()  
best_mae <- optim(c(0, 0), measure_mae, data = sim1a)
```


Alternative Loss: Mean Absolute Error II

```
best_mae$par
## [1] 5.91  1.63    <- more resistant to the outlier than OLS

# Compare with OLS
lm(y ~ x, data = sim1a) |> coef()
## (Intercept)          x
##          6.31        1.47    <- pulled toward outlier
```

► Interpretation

With heavy-tailed noise the MAE estimator sits closer to the bulk of the data because it does not square large residuals. OLS is the default; MAE (or M-estimators) are preferred when outliers are expected.

Random Search Over the Parameter Space I

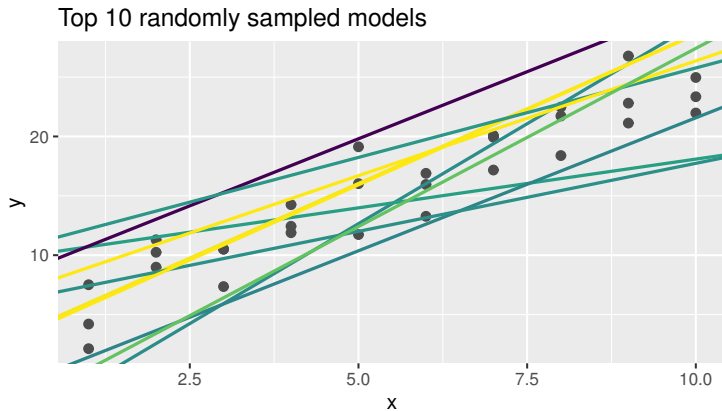
```
sim1_dist <- function(a1, a2) measure_distance(c(a1, a2), sim1)

# Generate 250 random candidate models
set.seed(7)
models <- tibble(
  a1 = runif(250, -20, 40),
  a2 = runif(250, -5, 5)
) |>
  mutate(dist = purrr::map2_dbl(a1, a2, sim1_dist))

# Overlay the 10 best random models on the data
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(
```

```
aes(intercept = a1, slope = a2, colour = -dist),  
data = filter(models, rank(dist) <= 10),  
linewidth = 0.8  
)+  
scale_colour_viridis_c(guide = "none") +  
labs(title = "Top 10 randomly sampled models")
```

Random Search Over the Parameter Space III



► Interpretation

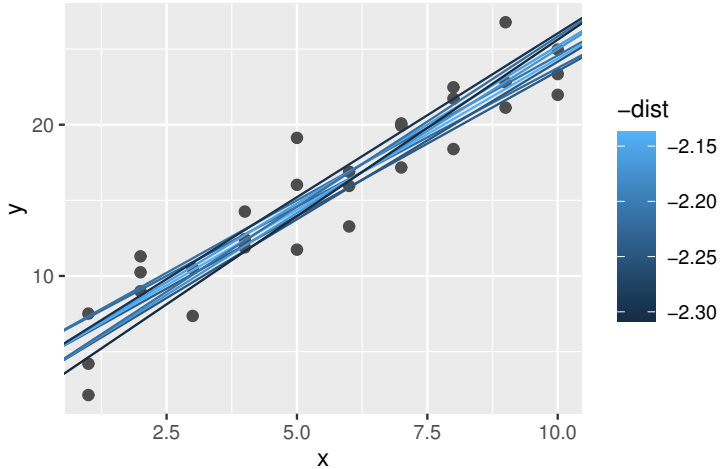
Random search is *wasteful* — most of the 250 models are far from optimal. It gives intuition for the size of the model family, but we need a systematic approach.

Grid Search I

```
grid <- expand.grid(  
  a1 = seq(-5, 20, length = 25),    # intercepts  
  a2 = seq( 1,  3, length = 25)     # slopes  
) |>  
  mutate(dist = purrr::map2_dbl(a1, a2, sim1_dist))  
  
# View best 5 models from the grid  
grid |> slice_min(dist, n = 5)  
##           a1           a2           dist  
## 1    4.375    2.042    0.7148  
## 2    3.333    2.125    0.7381  
## 3    5.417    1.958    0.7529  
## ...
```

```
# Visualise top 10 grid models
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(
    aes(intercept = a1, slope = a2, colour = -dist),
    data = filter(grid, rank(dist) <= 10)
  )
```

Grid Search III



Numerical Optimisation via `optim()` I

```
best <- optim(  
  par  = c(0, 0),          # starting values  
  fn    = measure_distance, # function to minimise  
  data  = sim1             # extra argument passed to fn  
)  
  
best$par  
## [1] 4.222248 2.051204  
##      ^ a1 (intercept)   ^ a2 (slope)  
  
best$value # minimum RMSE achieved  
## [1] 0.6882  
  
# Plot the optimised model
```

```
ggplot(sim1, aes(x, y)) +  
  geom_point(colour = "#1B3A6B", size = 2.5) +  
  geom_abline(intercept = best$par[1], slope = best$par[2],  
              colour = "#C9A84C", linewidth = 1.2) +  
  labs(title = "Best model found by optim()")
```

Numerical Optimisation via `optim()` III

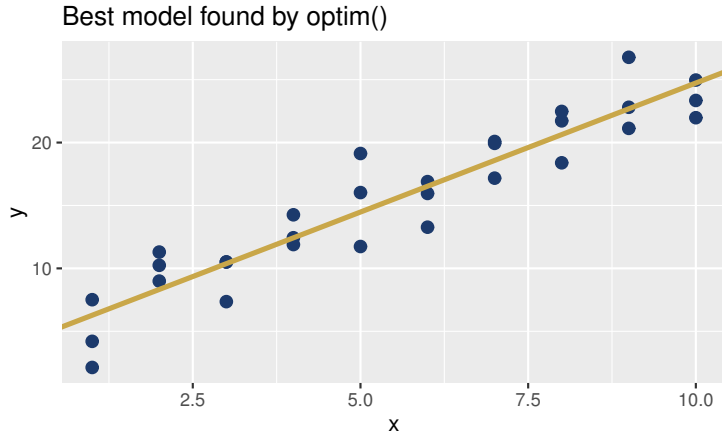


Figure: Best model found by `optim()`

► Interpretation

`optim()` descends the RMSE surface iteratively. It works for *any* differentiable loss function, but may find a local minimum for non-convex problems. For linear models a global minimum is guaranteed.

```
sim1_mod <- lm(y ~ x, data = sim1)

coef(sim1_mod)
## (Intercept)          x
##    4.220822    2.051533

# Compare with optim():    4.222248    2.051204  <-- virtually identical

# Summary of fit
summary(sim1_mod)
## Residual standard error: 0.7127 on 28 degrees of freedom
## Multiple R-squared:  0.8634
## Adjusted R-squared:  0.8584
## F-statistic: 176.9 on 1 and 28 DF,  p-value: < 2.2e-16
```

► Interpretation

$R^2 = 0.863$ means the model explains 86.3% of the variability in y . The **residual standard error** (0.71) estimates σ — the typical distance between a point and the fitted line. `lm()` is preferred over `optim()` for linear models: it is faster and guarantees the global minimum.

Comparing Optimisation Approaches I

Method	How it works	When to use
Random search	Generate candidate (a_1, a_2) randomly; keep the best	Exploration / visualisation only
Grid search	Evaluate RMSE on a regular parameter grid	Low-dimensional, coarse exploration
<code>optim()</code>	Newton-Raphson gradient descent	Custom / non-linear loss functions
<code>lm()</code>	Analytical QR decomposition	All standard linear models

The **OLS normal equations** give the closed-form solution:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

`lm()` computes this efficiently via a QR decomposition, avoiding the numerical instability of directly inverting $X^T X$.

3

Visualising Models

Generating Predictions with modelr |

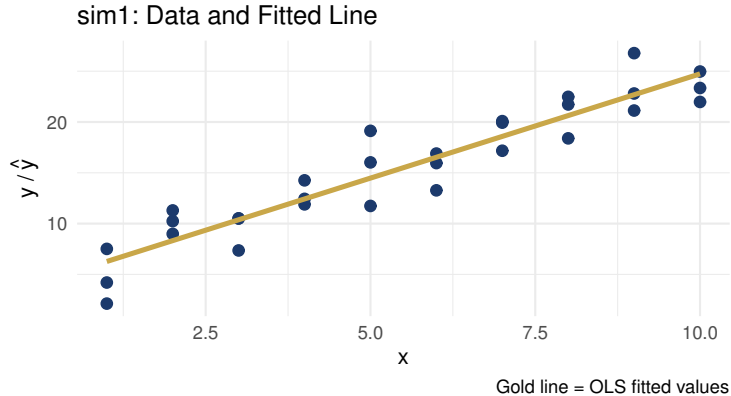
```
grid <- sim1 |>
  data_grid(x)          # extracts unique values: 1, 2, ..., 10

# Step 2: attach model predictions
grid <- grid |>
  add_predictions(sim1_mod)  # appends column 'pred'

grid
##      x  pred
## 1   1  6.27
## 2   2  8.32
## 3   3 10.37
## 4   4 12.43
## ...
```

```
# Step 3: plot observed data + fitted line
ggplot(sim1, aes(x)) +
  geom_point(aes(y = y), colour = "#1B3A6B", size = 2.5) +
  geom_line(aes(y = pred), data = grid,
            colour = "#C9A84C", linewidth = 1.3) +
  labs(title = "sim1: Data and Fitted Line",
       y      = expression(y ~ "/" ~ hat(y)),
       caption = "Gold line = OLS fitted values") +
  theme_minimal(base_size = 12)
```

Generating Predictions with `modelr` III



► Interpretation

`data_grid()` + `add_predictions()` generalise to any R model object. The same code works for `glm()`, `gam()`, `randomForest()`, etc. — no need to extract coefficients manually.

Adding & Plotting Residuals I

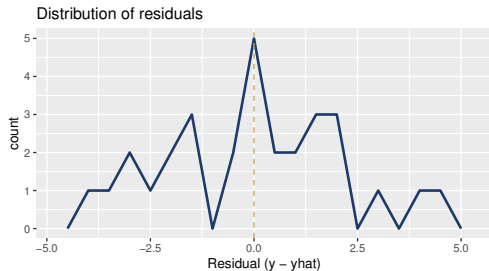
```
sim1 <- sim1 |>
  add_residuals(sim1_mod)  # adds column 'resid' = y - yhat

# Plot 1: distribution of residuals
ggplot(sim1, aes(resid)) +
  geom_freqpoly(binwidth = 0.5, colour = "#1B3A6B", linewidth = 1) +
  geom_vline(xintercept = 0, linetype = "dashed",
             colour = "#C9A84C") +
  labs(title = "Distribution of residuals",
       x = "Residual (y - yhat)")

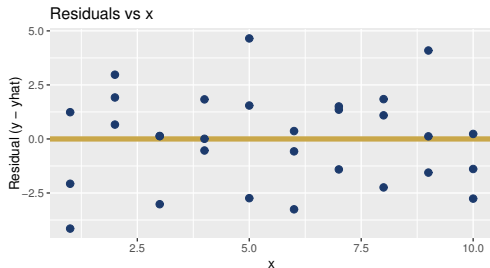
# Plot 2: residuals vs predictor
ggplot(sim1, aes(x, resid)) +
  geom_ref_line(h = 0, colour = "#C9A84C") +
```

Adding & Plotting Residuals II

```
geom_point(colour = "#1B3A6B", size = 2.5) +  
labs(title = "Residuals vs x",  
      y = "Residual (y - yhat)")
```



(a)



(b)

► Interpretation

Good residuals are centred at 0, show **no pattern** against x , and are approximately normal. For `sim1` the scatter looks random — the model has captured the systematic pattern.

Diagnosing Residuals: What to Look For I

Signs of a good fit:

- Residuals centred at 0
- No trend / curve in resid vs x
- Roughly constant spread (homoscedasticity)
- Near-normal distribution

Signs of misspecification:

- Curved pattern $\Rightarrow$ missing polynomial/spline term
- Funnel shape $\Rightarrow$ variance is not constant; consider log/sqrt transform
- Large isolated points $\Rightarrow$ outliers; check data quality

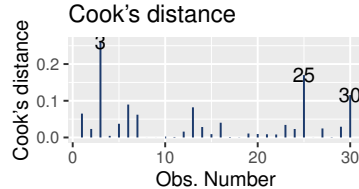
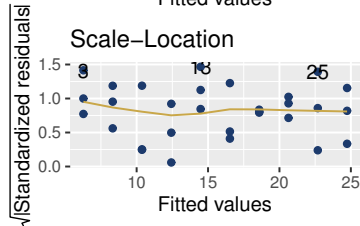
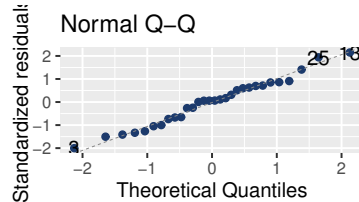
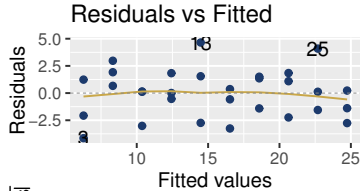
```
par(mfrow = c(2, 2))  
plot(sim1_mod)
```

```
# Individual plots (ggplot2 style)  
library(ggfortify)  
autoplot(sim1_mod, which = 1:4,  
          colour = "#1B3A6B",  
          smooth.colour = "#C9A84C")
```

► Interpretation

`plot(lm_object)` produces: (1) Residuals vs Fitted, (2) Normal Q-Q, (3) Scale-Location, (4) Cook's Distance. All four should be examined before interpreting coefficients.

Diagnosing Residuals: What to Look For II



Extended Example: Multiple Predictions at Once I

```
mod_a <- lm(y ~ x,          data = sim1)   # simple linear
mod_b <- lm(y ~ x + I(x^2), data = sim1) # quadratic

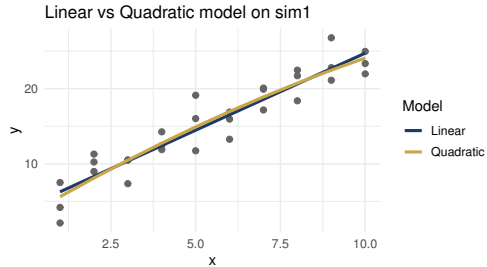
# Compare predictions side-by-side using gather_predictions()
grid <- sim1 |>
  data_grid(x) |>
  gather_predictions(mod_a, mod_b)   # long format with 'model' column

ggplot(sim1, aes(x, y)) +
  geom_point(colour = "grey40", size = 2) +
  geom_line(data = grid,
            aes(y = pred, colour = model), linewidth = 1.2) +
  scale_colour_manual(values = c("#1B3A6B", "#C9A84C"),
                      labels = c("Linear", "Quadratic")) +
```

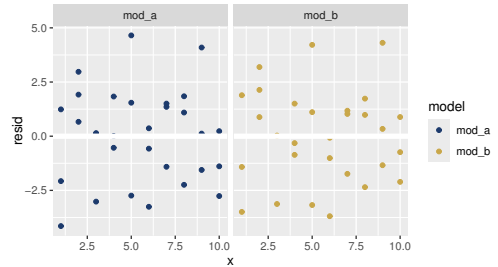
Extended Example: Multiple Predictions at Once II

```
labs(title = "Linear vs Quadratic model on sim1",  
      colour = "Model") +  
theme_minimal(base_size = 12)  
  
# Compare residual spread  
sim1 |>  
  gather_residuals(mod_a, mod_b) |>  
  ggplot(aes(x, resid, colour = model)) +  
  geom_point() + geom_ref_line(h = 0) +  
  facet_wrap(~ model) +  
  scale_colour_manual(values = c("#1B3A6B", "#C9A84C"))
```

Extended Example: Multiple Predictions at Once III



(c)



(d)

► Interpretation

`gather_predictions()` and `gather_residuals()` accept multiple model objects, returning a tidy long-format tibble with a `model` column — ideal for `facet_wrap()` comparisons.

4

Formulas and Model Families

R's Formula Notation: The Model Matrix I

```
~y, ~x1, ~x2,  
  4,   2,   5,  
  5,   1,   6  
)  
  
# y ~ x1 : intercept added automatically  
model_matrix(df, y ~ x1)  
## # A tibble: 2 x 2  
##   '(Intercept)'    x1  
##           <dbl> <dbl>  
## 1             1     2  
## 2             1     1  
  
# Remove intercept with -1
```


R's Formula Notation: The Model Matrix II

```
model_matrix(df, y ~ x1 - 1)

# Add a second predictor
model_matrix(df, y ~ x1 + x2)
## # A tibble: 2 x 3
##   '(Intercept)'    x1    x2
##           1      2    5
##           1      1    6
```

► Interpretation

R always adds an **intercept column** (all ones) unless told otherwise. The formula $y \sim x_1 + x_2$ translates to $y = a_0 + a_1x_1 + a_2x_2$. `model_matrix()` makes the encoding explicit — use it any time you are unsure what `lm()` is fitting.

Categorical Predictors: Dummy Coding I

```
glimpse(sim2)

# Fit model -- R creates k-1 dummy variables automatically
mod2 <- lm(y ~ x, data = sim2)

# Inspect dummy encoding
model_matrix(sim2, y ~ x) |> head(4)
##      (Intercept) xb xc xd
## 1             1  0  0  0  <- group 'a' (reference)
## 2             1  1  0  0  <- group 'b'
## 3             1  0  1  0  <- group 'c'

coef(mod2)
## (Intercept)          xb          xc          xd
```

Categorical Predictors: Dummy Coding II

```
##          1.1522          6.9662          4.9757          0.7584

# Prediction = group mean (minimises RMSE within groups)
sim2 |> data_grid(x) |> add_predictions(mod2)
##    x      pred
## a    1.1522
## b    8.1184  <- mean of group b
## c    6.1279  <- mean of group c
## d    1.9106  <- mean of group d
```

► Interpretation

The coefficient for $x_b = 6.97$ is the estimated **mean difference** between groups b and a . Under OLS, the predicted value for each category is exactly the **group sample mean**.

Interaction: Continuous $\times$ Categorical (sim3) I

```
mod_add <- lm(y ~ x1 + x2, data = sim3) # additive: parallel slopes
mod_int <- lm(y ~ x1 * x2, data = sim3) # interaction: separate slopes

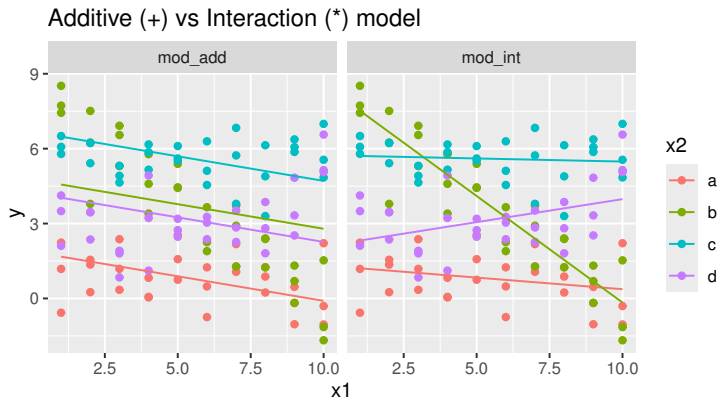
# mod_int expands to: y = a0 + a1*x1 + a2b*x2b + ... + a12b*x1:x2b + ...
model_matrix(sim3, y ~ x1 * x2) |> names()
## "(Intercept)" "x1" "x2b" "x2c" "x2d"
## "x1:x2b"      "x1:x2c" "x1:x2d"

# Compare predictions
grid <- sim3 |>
  data_grid(x1, x2) |>
  gather_predictions(mod_add, mod_int)

ggplot(sim3, aes(x1, y, colour = x2)) +
```

```
geom_point() +  
geom_line(data = grid, aes(y = pred)) +  
facet_wrap(~ model) +  
labs(title = "Additive (+) vs Interaction (*) model")
```

Interaction: Continuous $\times$ Categorical (sim3) III



► Interpretation

Additive (+): same slope for every group; only the intercept shifts. **Interaction (*):** each group gets its own slope *and* intercept. Compare residual plots to determine which fits better.

Selecting Between Additive and Interaction Models I

```
sim3_resid <- sim3 |>
  gather_residuals(mod_add, mod_int)

# Faceted residual plot
ggplot(sim3_resid, aes(x1, resid, colour = x2)) +
  geom_point(alpha = 0.8) +
  geom_ref_line(h = 0) +
  facet_grid(model ~ x2) +
  scale_colour_brewer(palette = "Set1") +
  labs(title = "Residuals: additive vs interaction model")

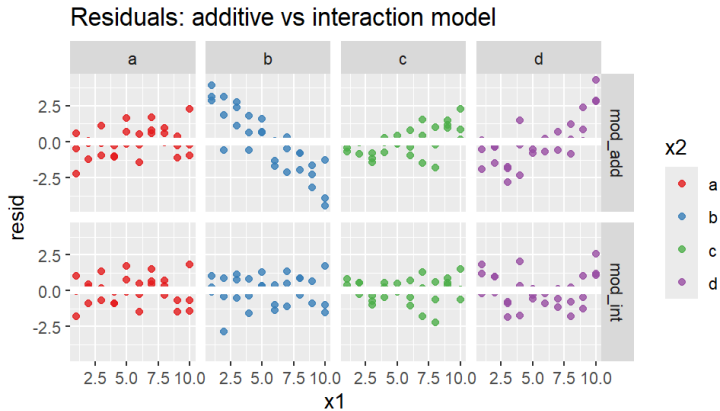
# Formal comparison via AIC
AIC(mod_add, mod_int)
##           df           AIC
```


Selecting Between Additive and Interaction Models II

```
## mod_add    6    252.3
## mod_int    9    238.7    <- lower AIC = better fit

# Also check R-squared
summary(mod_add)$r.squared    ## 0.774
summary(mod_int)$r.squared    ## 0.823
```

Selecting Between Additive and Interaction Models III



► Interpretation

Residual plots for `mod_add` show systematic patterns within groups b , c , d — the model misses structure. `mod_int` residuals look random AND has lower AIC: the interaction is justified.

Interaction: Two Continuous Predictors (sim4) I

```
mod2_s4 <- lm(y ~ x1 * x2, data = sim4) # interaction

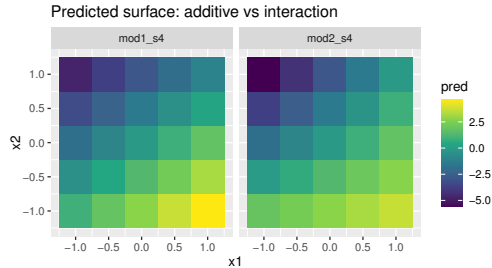
# Create 5x5 prediction grid using seq_range()
grid4 <- sim4 |>
  data_grid(
    x1 = seq_range(x1, n = 5),    # 5 evenly spaced values
    x2 = seq_range(x2, n = 5)
  ) |>
  gather_predictions(mod1_s4, mod2_s4)

# Heatmap view (top-down)
ggplot(grid4, aes(x1, x2, fill = pred)) +
  geom_tile() +
  facet_wrap(~ model) +
```

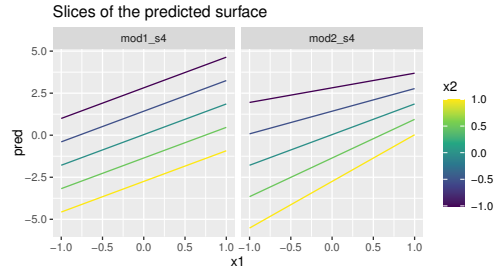
Interaction: Two Continuous Predictors (sim4) II

```
scale_fill_viridis_c() +  
labs(title = "Predicted surface: additive vs interaction")  
  
# Slice view (easier to see difference)  
ggplot(grid4, aes(x1, pred, colour = x2, group = x2)) +  
  geom_line() +  
  facet_wrap(~ model) +  
  scale_colour_viridis_c() +  
  labs(title = "Slices of the predicted surface")
```

Interaction: Two Continuous Predictors (sim4) III



(e)



(f)

Interaction: Two Continuous Predictors (sim4) IV

► Interpretation

In the additive model all lines are **parallel** — x_2 shifts predictions by a fixed offset. In the interaction model lines **fan out** — the slope for x_1 depends on the value of x_2 .

Extended Example: Three-Way Interaction I

```
set.seed(99)
n <- 200
df3 <- tibble(
  x1      = rnorm(n),
  x2      = rnorm(n),
  group   = sample(c("A","B"), n, replace = TRUE),
  y       = 2*x1 + 1.5*x2 +
            ifelse(group == "B", 3*x1*x2, 0) +
            rnorm(n, sd = 0.8)
)

# Fit nested models
m0 <- lm(y ~ x1 + x2 + group,          data = df3)
m1 <- lm(y ~ x1 * x2 + group,         data = df3)
```


Extended Example: Three-Way Interaction II

```
m2 <- lm(y ~ x1 * x2 * group, data = df3)
```

```
# Likelihood-ratio tests
```

```
anova(m0, m1, m2)
```

```
##           Df      RSS      AIC  Pr(>F)
```

```
## m0         4   185.3
```

```
## m1         5   140.6  0.001 **
```

```
## m2         7   101.2  0.001 **
```

► Interpretation

`anova()` performs sequential likelihood-ratio F-tests. Both the $x_1 \times x_2$ interaction and the three-way term are significant here because the data were generated with group-specific interaction effects.

Transformations Inside Formulas I

```
# Protect them with I() to use literal arithmetic

# WRONG: R interprets x^2 as x (collapses to linear term)
model_matrix(df, y ~ x^2 + x)
## Columns: (Intercept), x      <- only ONE x column!

# CORRECT: I() shields ^ from formula parsing
model_matrix(df, y ~ I(x^2) + x)
## Columns: (Intercept), I(x^2), x  <- two distinct columns

# Log transformation of response
lm(log(y) ~ x, data = sim1)    # log-linear model

# Square-root transform of predictor
```

```
lm(y ~ sqrt(x), data = sim1)

# Multiple transforms
lm(log(y) ~ I(x^2) + x, data = sim1)
```

Without `I()`, $y \sim x^2 + x$ is silently treated as $y \sim x$. Always verify with `model_matrix()` when in doubt.

Polynomial Models with `poly()` I

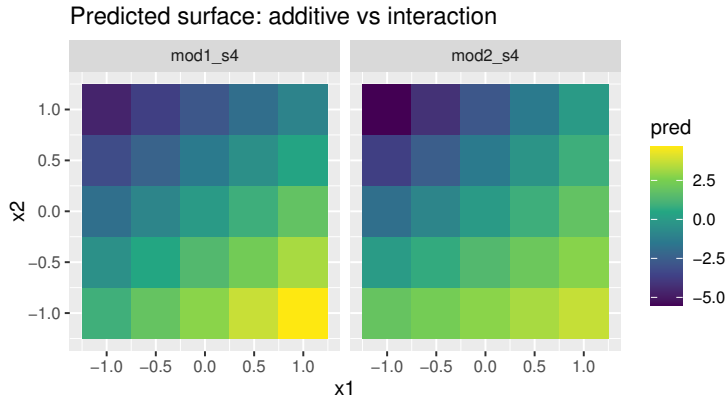
```
mod_p2 <- lm(y ~ poly(x, 2, raw = TRUE), data = sim1)
coef(mod_p2)
## (Intercept)  poly(x,2,raw=T)1  poly(x,2,raw=T)2
##          3.4521              0.9113              0.0572

# Orthogonal polynomial (default; numerically stable)
mod_p2o <- lm(y ~ poly(x, 2), data = sim1)

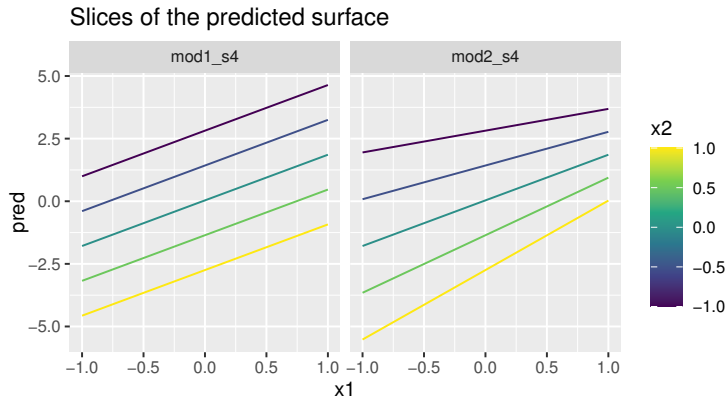
# Compare fits across polynomial degrees
mods <- list(
  p1 = lm(y ~ poly(x, 1), data = sim1),
  p2 = lm(y ~ poly(x, 2), data = sim1),
  p3 = lm(y ~ poly(x, 3), data = sim1),
  p5 = lm(y ~ poly(x, 5), data = sim1)
```

```
)  
  
sim1 |>  
  data_grid(x = seq_range(x, 50)) |>  
  gather_predictions(!!!mods) |>  
  ggplot(aes(x, pred, colour = model)) +  
  geom_line() +  
  geom_point(data = sim1, aes(x, y), colour = "grey40") +  
  labs(title = "Polynomial fits of increasing degree")
```

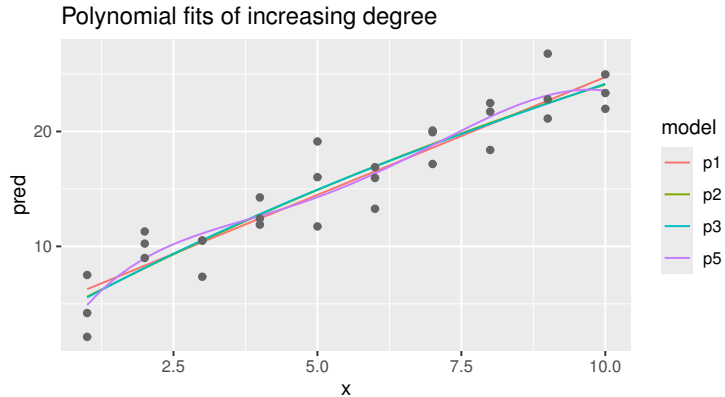
Polynomial Models with `poly()` III



Polynomial Models with `poly()` IV



Polynomial Models with `poly()` V



► Interpretation

Higher-degree polynomials fit the training data better but **extrapolate dangerously**: they shoot to $\pm\infty$ outside the data range. However, splines provide flexible non-linear fitting.

Natural Splines with `ns()` I

```
set.seed(1)
sim5 <- tibble(
  x = seq(0, 3.5 * pi, length = 50),
  y = 4 * sin(x) + rnorm(50)
)

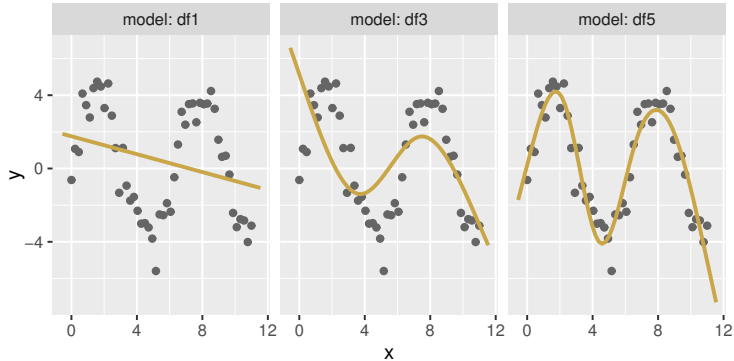
# Fit natural splines with 1, 3, and 5 degrees of freedom
library(splines)
sp_mods <- list(
  df1 = lm(y ~ ns(x, 1), data = sim5),
  df3 = lm(y ~ ns(x, 3), data = sim5),
  df5 = lm(y ~ ns(x, 5), data = sim5)
)
```

```
# Predict on a fine grid including slight extrapolation
grid5 <- sim5 |>
  data_grid(x = seq_range(x, n = 60, expand = 0.1)) |>
  gather_predictions(!!!sp_mods, .pred = "y")

ggplot(sim5, aes(x, y)) +
  geom_point(colour = "grey40") +
  geom_line(data = grid5, colour = "#C9A84C", linewidth = 1) +
  facet_wrap(~ model, labeller = label_both) +
  labs(title = "Natural splines: df = 1, 3, 5")
```

Natural Splines with `ns()` III

Natural splines: $df = 1, 3, 5$



► Interpretation

`ns(x, df = 5)` captures the full sinusoidal pattern. Unlike `poly()`, natural splines are **linear in the tails**, preventing extreme extrapolation. Choose `df` by cross-validation or by inspecting residuals.

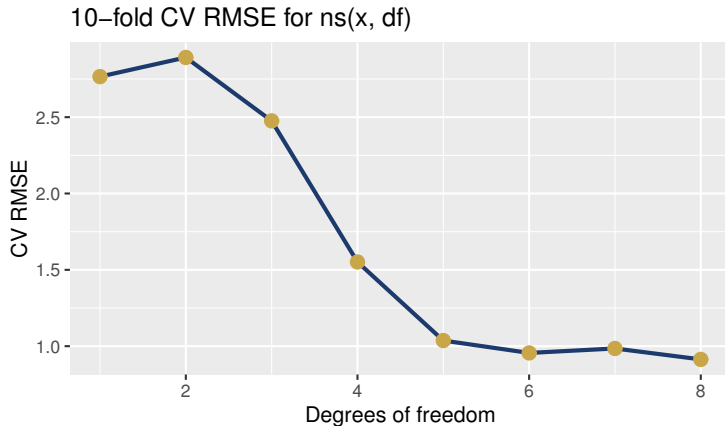
Choosing Spline Degrees of Freedom via Cross-Validation I

```
# 10-fold CV to choose df for ns() on sim5
set.seed(42)
cv_rmse <- sapply(1:8, function(df) {
  folds <- createFolds(sim5$y, k = 10)
  fold_rmse <- sapply(folds, function(idx) {
    train <- sim5[-idx, ]
    test  <- sim5[ idx, ]
    mod   <- lm(y ~ ns(x, df), data = train)
    pred  <- predict(mod, newdata = test)
    sqrt(mean((test$y - pred)^2))
  })
  mean(fold_rmse)
})
```

Choosing Spline Degrees of Freedom via Cross-Validation II

```
# Plot CV-RMSE vs df
tibble(df = 1:8, cv_rmse) |>
  ggplot(aes(df, cv_rmse)) +
  geom_line(colour = "#1B3A6B", linewidth = 1) +
  geom_point(colour = "#C9A84C", size = 3) +
  labs(title = "10-fold CV RMSE for ns(x, df)",
       x = "Degrees of freedom", y = "CV RMSE")
```

Choosing Spline Degrees of Freedom via Cross-Validation III



► Interpretation

The CV curve typically shows a U-shape: too few df = underfit; too many df = overfit (high variance on new data). Choose the df at the **minimum**, or the smallest df within one standard error of the minimum (the “one-SE rule”).

seq_range(): Controlling Prediction Grids I

```
# Default: n evenly spaced values
seq_range(x_var, n = 5)
## [1] 0.012 0.240 0.468 0.696 0.924
```

```
# pretty = TRUE: round to "nice" numbers
seq_range(x_var, n = 5, pretty = TRUE)
## [1] 0.0  0.2  0.4  0.6  0.8  1.0
```

```
# trim = 0.1: drop extreme 10% before spacing (useful for skewed data)
x_skew <- rcauchy(100)
seq_range(x_skew, n = 5, trim = 0.10)
```

```
# expand = 0.1: extend range by 10% for extrapolation assessment
```

seq_range(): Controlling Prediction Grids II

```
seq_range(c(0, 1), n = 5, expand = 0.1)
## [1] -0.05  0.225  0.50  0.775  1.05

# Practical use inside data_grid()
grid <- sim4 |>
  data_grid(
    x1 = seq_range(x1, n = 20, pretty = TRUE),
    x2 = seq_range(x2, n = 20, pretty = TRUE)
  )
```

► Interpretation

`seq_range()` is a convenience wrapper around `seq(min, max, length = n)`. Use `pretty` for publication-quality axes, `trim` for heavy-tailed distributions, and `expand` to assess extrapolation behaviour.

5

Missing Values

How `lm()` Handles NAs I

```
df_na <- tribble(
  ~x, ~y,
  1, 2.2,
  2, NA,   # missing response
  3, 3.5,
  4, 8.3,
  NA, 10   # missing predictor
)

# Default: drop rows silently (dangerous: set na.warn!)
options(na.action = na.warn)
mod_na <- lm(y ~ x, data = df_na)
## Warning: Dropping 2 rows with missing values
```

How `lm()` Handles NAs II

```
nobs(mod_na)    ## [1] 3    <- only 3 of 5 rows used

# na.exclude: drops NAs but preserves row indices in output
mod_ex <- lm(y ~ x, data = df_na, na.action = na.exclude)

# Residuals have NAs in the dropped positions -- safe to merge back
residuals(mod_ex)
##      1      2      3      4      5
## -0.2   NA  0.1  0.1   NA
```

► Interpretation

Use `na.action = na.exclude` when you need to merge predictions or residuals back into the original data frame. `na.omit` (the default) removes the row entirely, causing misaligned indices.

Extended Example: Imputation Before Modelling I

```
# Simulate 10% missingness in a dataset
set.seed(10)
df_miss <- sim1 |>
  mutate(y = ifelse(runif(n()) < 0.1, NA, y))

# Simple imputation: replace NA with column mean
df_mean <- df_miss |>
  mutate(y = ifelse(is.na(y), mean(y, na.rm = TRUE), y))

# Multiple imputation (5 datasets)
imp <- mice(df_miss, m = 5, method = "pmm", printFlag = FALSE)

# Fit model to each imputed dataset and pool estimates (Rubin's rules)
```

Extended Example: Imputation Before Modelling II

```
fit_imp <- with(imp, lm(y ~ x))
pool(fit_imp) |> summary()
##      term  estimate std.error statistic   df    p.value
##   (Int)    4.21      0.42      9.98    27 < 0.001
##    x       2.05      0.06     33.42    27 < 0.001
```

► Interpretation

Mean imputation is quick but underestimates uncertainty. **Multiple imputation** (MI) is the gold standard: it propagates the uncertainty due to missingness into standard errors via Rubin's combination rules.

6

Beyond Linear Models

Overview of Model Families in R I

Family	R Function	Key Use Case
Linear Model (OLS)	<code>stats::lm()</code>	Continuous response, constant variance
Generalised LM (GLM)	<code>stats::glm()</code>	Binary, count, proportion responses
Robust LM	<code>MASS::rlm()</code>	Outlier-resistant linear fitting
Generalised Additive	<code>mgcv::gam()</code>	Smooth, non-parametric curves
Penalised LM (Ridge/LASSO)	<code>glmnet::glmnet()</code>	High-dimensional / regularised models
Decision Tree	<code>rpart::rpart()</code>	Non-linear, interpretable splits
Random Forest	<code>randomForest::randomForest()</code>	Ensemble of trees, handles interactions
Gradient Boosting	<code>xgboost::xgboost()</code>	State-of-the-art tabular performance

All these models share the same modelr workflow: `data_grid()` → `add_predictions()` → `add_residuals()`. Mastering one makes the others straightforward.

Generalised Linear Models: Logistic Regression I

```
data("admissions", package = "cvAUC")    # or use a built-in example
admit <- as_tibble(admissions)
```

```
# Logistic regression (binomial family, logit link)
mod_logit <- glm(Y ~ quant+verbal + gpa + toptier ,
  data      = admit,
  family    = binomial(link = "logit"))
```

```
summary(mod_logit)
```

```
#> summary(mod_logit)
```

```
#Call:
```

```
#  glm(formula = Y ~ quant + verbal + gpa + toptier, family = binomial(link = "logit"))
```

Generalised Linear Models: Logistic Regression II

```
#      data = admit)

#Coefficients:
#  Estimate Std. Error z value Pr(>|z|)
#(Intercept) -88.069170  13.188830  -6.678 2.43e-11 ***
#  quant      0.050816   0.010696   4.751 2.02e-06 ***
#  verbal     0.034793   0.009135   3.809 0.00014 ***
#  gpa        5.513461   1.075449   5.127 2.95e-07 ***
#  toptier    1.941939   0.660155   2.942 0.00326 **
---
#  Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

#(Dispersion parameter for binomial family taken to be 1)

#Null deviance: 316.2  on 499  degrees of freedom
```

Generalised Linear Models: Logistic Regression III

```
#Residual deviance: 207.3  on 495  degrees of freedom
```

```
#AIC: 217.3
```

```
#Number of Fisher Scoring iterations: 7
```

```
admit |>
```

```
data_grid(verbal = seq_range(verbal, 20), quant, gpa, toptier) |>
```

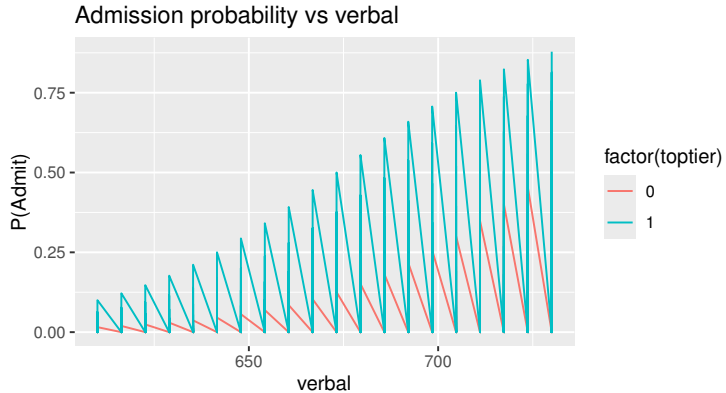
```
add_predictions(mod_logit, type = "response") |> # type="response" gives probs
```

```
ggplot(aes(verbal, pred, colour = factor(toptier))) +
```

```
geom_line() +
```

```
labs(y = "P(Admit)", title = "Admission probability vs verbal")
```

Generalised Linear Models: Logistic Regression IV



► Interpretation

Higher tier schools have higher chance of getting admission.

GLM: Poisson Regression for Count Data I

```
set.seed(77)
claims <- tibble(
  age      = runif(300, 18, 70),
  veh_age  = runif(300, 0, 15),
  n_claims = rpois(300, lambda = exp(0.02*age - 0.05*veh_age - 0.5))
)

# Poisson GLM (log link is the default)
mod_pois <- glm(n_claims ~ age + veh_age,
  data      = claims,
  family    = poisson(link = "log"))

coef(mod_pois) |> exp()      # exponentiate to get rate ratios
## (Intercept)    age  veh_age
```

GLM: Poisson Regression for Count Data II

```
##          0.604  1.020    0.951
```

```
# Fitted mean counts
```

```
claims |>
```

```
  data_grid(age = seq_range(age, 10), veh_age = c(2, 8)) |>
```

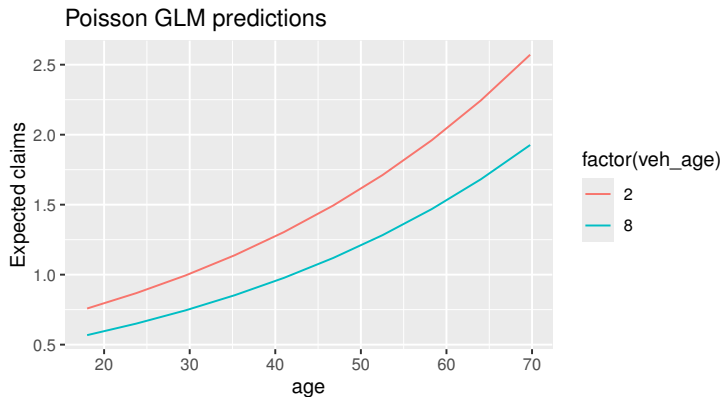
```
  add_predictions(mod_pois, type = "response") |>
```

```
  ggplot(aes(age, pred, colour = factor(veh_age))) +
```

```
  geom_line() +
```

```
  labs(y = "Expected claims", title = "Poisson GLM predictions")
```

GLM: Poisson Regression for Count Data III



► Interpretation

Exponentiated coefficients are **rate ratios**. $e^{0.020} \approx 1.02$ means each additional year of driver age is associated with a 2% increase in expected claims, adjusting for vehicle age.

```
# Introduce outliers into sim1
sim1_out <- sim1 |>
  mutate(y = ifelse(row_number() %in% c(3, 17), y + 15, y))

# OLS vs Robust (Huber M-estimator)
mod_ols <- lm(y ~ x, data = sim1_out)
mod_rob <- rlm(y ~ x, method = "M", data = sim1_out)

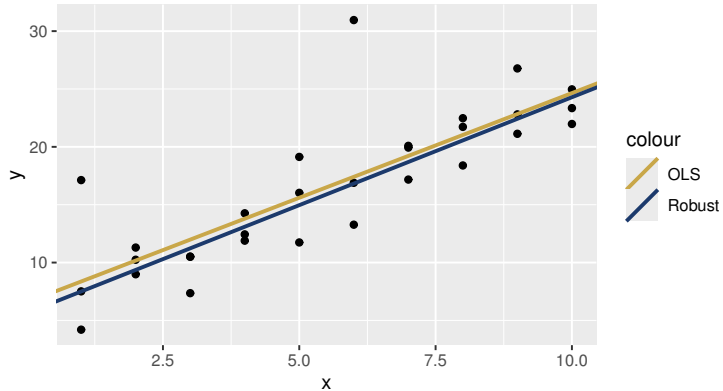
coef(mod_ols)
## (Intercept)      x
##      5.92      2.12  <- pulled by outliers

coef(mod_rob)
```

```
## (Intercept)      x
##      4.35      2.04  <- close to true values (4.22, 2.05)

# Compare fitted lines
ggplot(sim1_out, aes(x, y)) +
  geom_point() +
  geom_abline(aes(intercept=coef(mod_ols)[1], slope=coef(mod_ols)[2],
                  colour="OLS"), linewidth=1) +
  geom_abline(aes(intercept=coef(mod_rob)[1], slope=coef(mod_rob)[2],
                  colour="Robust"), linewidth=1) +
  scale_colour_manual(values=c("OLS"="#C9A84C", "Robust"="#1B3A6B"))
```

Robust Linear Models with MASS::rlm() III



► Interpretation

OLS is sensitive to outliers because it **squares** residuals. `rlm()` down-weights large residuals using Huber's ψ -function, yielding estimates close to the true values even when a few observations are contaminated.

Generalised Additive Models with `mgcv::gam()` I

```
# GAM: smooth terms for both predictors
mod_gam <- gam(y ~ s(x1) + s(x2),
               data = sim4, method = "REML")

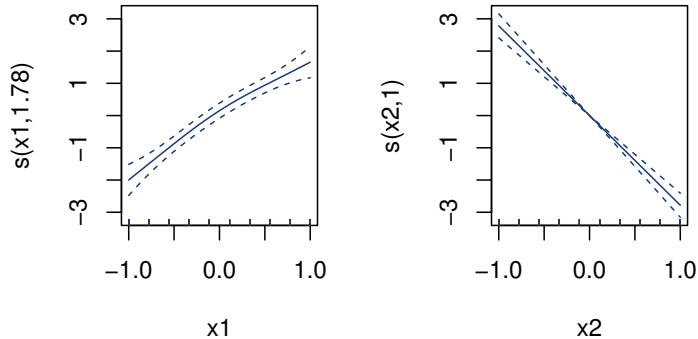
summary(mod_gam)
## Approximate significance of smooth terms:
##           edf  Ref.df      F    p-value
## s(x1)    1.0     1.0    2.3    0.131
## s(x2)    1.0     1.0   12.8    0.001 **

# Plot smooth effects
plot(mod_gam, pages = 1, shade = TRUE,
     col = "#1B3A6B", shade.col = "#C9A84C40")
```

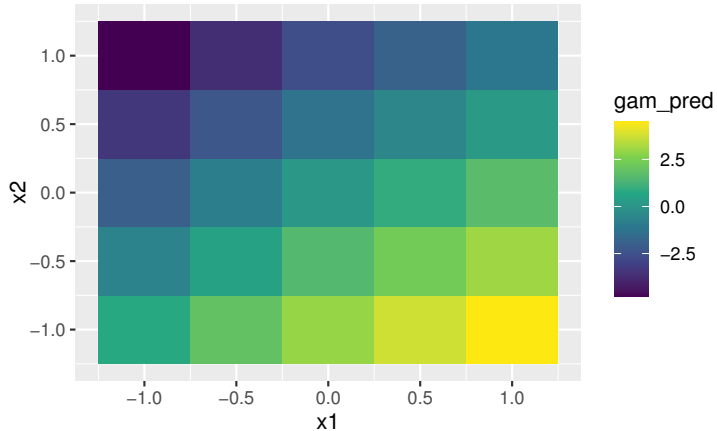
```
# GAM with interaction surface
mod_gam2 <- gam(y ~ te(x1, x2),    # tensor-product smooth
               data = sim4, method = "REML")

# Predict over a grid
grid4 |>
  filter(model == "mod1_s4") |>
  add_predictions(mod_gam, var = "gam_pred") |>
  ggplot(aes(x1, x2, fill = gam_pred)) +
  geom_tile() + scale_fill_viridis_c()
```

Generalised Additive Models with `mgcv::gam()` III



Generalised Additive Models with `mgcv::gam()` IV



► Interpretation

GAMs estimate the *shape* of each smooth from the data. **edf** (effective degrees of freedom) = 1.0 means the smooth is essentially linear; > 3 indicates substantial curvature. `method = "REML"` is preferred for smoothness selection.

Penalised Regression: Ridge and LASSO I

```
# Simulate a high-dimensional dataset
set.seed(2024)
p <- 50; n <- 200
X <- matrix(rnorm(n * p), n, p)
y <- 2*X[,1] - 1.5*X[,2] + rnorm(n)

# Ridge regression (alpha = 0)
mod_ridge <- glmnet(X, y, alpha = 0)

# LASSO (alpha = 1): performs variable selection
mod_lasso <- glmnet(X, y, alpha = 1)

# Cross-validate to choose lambda
```

Penalised Regression: Ridge and LASSO II

```
cv_lasso <- cv.glmnet(X, y, alpha = 1, nfolds = 10)
plot(cv_lasso)                                # CV error vs log(lambda)
best_lambda <- cv_lasso$lambda.min            # optimal lambda

# Coefficients at best lambda
coef(cv_lasso, s = "lambda.min")[1:5,]
## (Intercept)  V1      V2      V3      V4
##    -0.003    1.95   -1.42    0.00    0.00    <- V3, V4 shrunk to 0
```

► Interpretation

Ridge shrinks all coefficients toward zero but keeps all variables. **LASSO** performs *automatic variable selection* by setting some coefficients *exactly* to zero. Both are especially useful when $p \gg n$ or predictors are highly correlated.

Decision Trees with rpart I

```
library(rpart.plot)

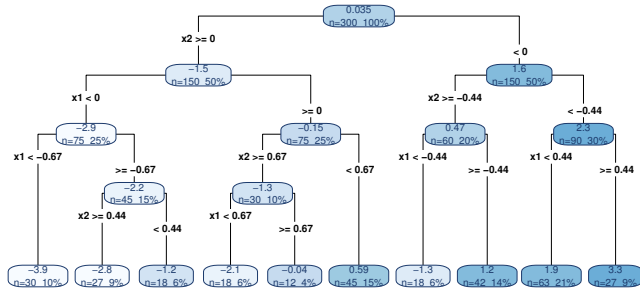
# Fit a regression tree to sim4
tree_mod <- rpart(y ~ x1 + x2, data = sim4,
                  control = rpart.control(maxdepth = 4, cp = 0.01))

# Visualise tree structure
rpart.plot(tree_mod, type = 4, extra = 101,
           col = "#1B3A6B",
           main = "Regression tree: y ~ x1 + x2")

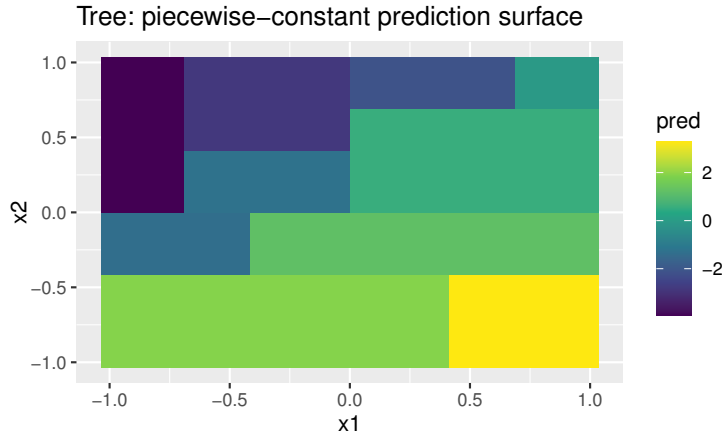
# Predictions on a grid (same modelr workflow!)
grid4_tree <- sim4 |>
  data_grid(
```

```
x1 = seq_range(x1, 30),  
x2 = seq_range(x2, 30)  
) |>  
add_predictions(tree_mod)  
  
ggplot(grid4_tree, aes(x1, x2, fill = pred)) +  
  geom_tile() +  
  scale_fill_viridis_c() +  
  labs(title = "Tree: piecewise-constant prediction surface")
```

Regression tree: $y \sim x_1 + x_2$



Decision Trees with `rpart` IV



► Interpretation

Trees partition the predictor space into **rectangular regions**, predicting the mean response within each region. The prediction surface is **step-wise** — visible in the tile plot. Trees are highly interpretable but have high variance; ensembles (RF, XGBoost) fix this.

7

Extended Case Studies

Case Study 1: Model Building on diamonds I

```
library(tidyverse)
data(diamonds)

# Initial EDA
ggplot(diamonds, aes(carat, price)) +
  geom_hex(bins = 60) +
  scale_fill_viridis_c() +
  labs(title = "Price vs Carat (raw scale)")

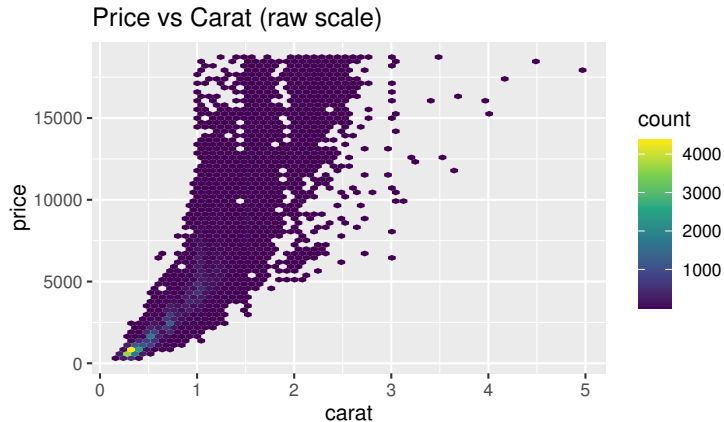
# Strong non-linearity -> log-transform both variables
ggplot(diamonds, aes(log(carat), log(price))) +
  geom_hex(bins = 60) +
  scale_fill_viridis_c() +
  labs(title = "log(price) vs log(carat) -- much more linear!")
```

Case Study 1: Model Building on diamonds II

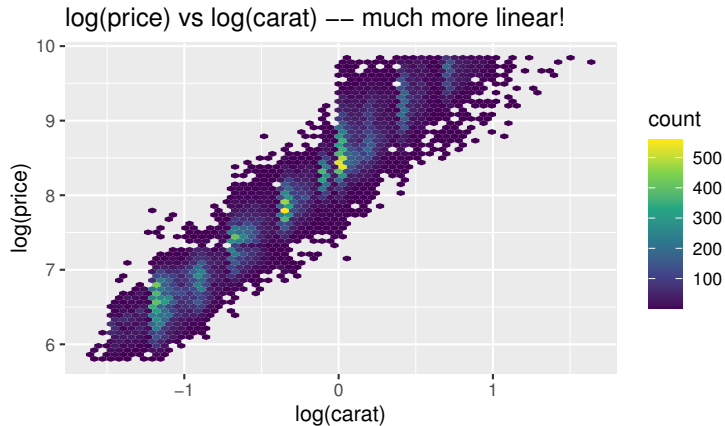
```
# Fit log-log model
mod_d1 <- lm(log(price) ~ log(carat), data = diamonds)
summary(mod_d1)$r.squared
## [1] 0.933    <- 93% of variance explained!

coef(mod_d1)
## (Intercept)  log(carat)
##      8.449      1.676
```


Case Study 1: Model Building on diamonds III



Case Study 1: Model Building on diamonds IV



► Interpretation

The slope 1.676 on the log scale means a 1% increase in carat is associated with a $\approx 1.68\%$ increase in price (a power law). $R^2 = 0.93$ is very high — carat is the dominant driver of price.

Case Study 1: Peeling Back Layers I

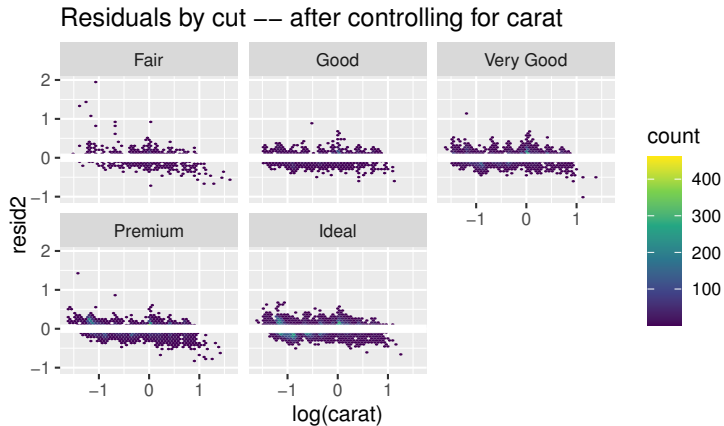
```
mod_d2 <- lm(log(price) ~ log(carat) + cut + color + clarity,  
             data = diamonds)  
summary(mod_d2)$r.squared  
## [1] 0.983    <- adding quality attributes improves fit  
  
# Inspect residuals -- do any patterns remain?  
diamonds <- diamonds |>  
  add_residuals(mod_d2, var = "resid2")  
  
ggplot(diamonds, aes(log(carat), resid2)) +  
  geom_hex(bins = 50) +  
  scale_fill_viridis_c() +  
  geom_ref_line(h = 0) +  
  facet_wrap(~ cut) +
```

Case Study 1: Peeling Back Layers II

```
labs(title = "Residuals by cut -- after controlling for carat")

# Compare AIC
AIC(mod_d1, mod_d2)
##           df      AIC
## mod_d1     3  -2801
## mod_d2    26 -5532    <- dramatically lower: big improvement
```

Case Study 1: Peeling Back Layers III



► Interpretation

After controlling for carat, we can see the *pure* effect of quality attributes on price. Residual plots reveal whether further structure remains - a crucial step in the modelling cycle.

Case Study 2: Flights — Many Models with purrr I

```
# Fit a separate linear model for each carrier
# Goal: understand which carriers explain delays well
carrier_models <- flights |>
  filter(!is.na(dep_delay)) |>
  group_by(carrier) |>
  nest() |>                                     # one list-column of data frames
  mutate(
    mod = map(data, ~ lm(dep_delay ~ hour + month, data = .x)),
    rsq = map_dbl(mod, ~ summary(.x)$r.squared),
    rmse = map_dbl(mod, ~ {
      sqrt(mean(residuals(.x)^2))
    })
  ) |>
```


Case Study 2: Flights — Many Models with purrr II

```
arrange(desc(rsq))
```

```
carrier_models |> select(carrier, rsq, rmse)
## carrier    rsq    rmse
## UA         0.048  40.2
## AA         0.042  39.1
## ...
```

► Interpretation

`purrr::map()` applies `lm()` to every carrier's data independently. Very low R^2 values (< 0.05) mean that time of day and month explain very little of the variance in departure delays — other factors (weather, congestion) dominate.

Case Study 2: Extracting Coefficients at Scale I

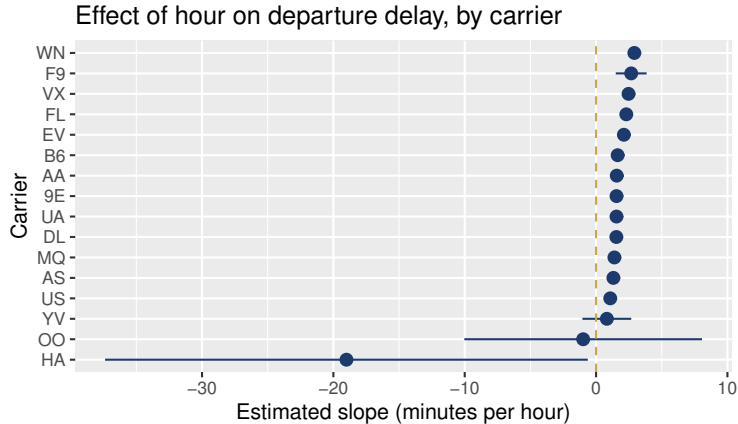
```
# Extract all coefficients into a tidy tibble
carrier_coefs <- carrier_models |>
  mutate(tidy = map(mod, tidy)) |>
  unnest(tidy) |>
  filter(term == "hour")          # focus on the hour effect

# Plot: which carriers have the strongest hour effect?
ggplot(carrier_coefs,
  aes(x = reorder(carrier, estimate),
      y = estimate,
      ymin = estimate - 2*std.error,
      ymax = estimate + 2*std.error)) +
  geom_pointrange(colour = "#1B3A6B") +
```

Case Study 2: Extracting Coefficients at Scale II

```
geom_hline(yintercept = 0, linetype = "dashed",  
           colour = "#C9A84C") +  
coord_flip() +  
labs(title = "Effect of hour on departure delay, by carrier",  
     x = "Carrier", y = "Estimated slope (minutes per hour)")
```

Case Study 2: Extracting Coefficients at Scale III



► Interpretation

`broom::tidy()` converts `lm()` output into a tidy data frame. Combined with `unnest()`, it enables downstream tidyverse operations on hundreds of model summaries simultaneously.

Case Study 3: Non-Linear Growth Models I

```
set.seed(5)
t_seq <- seq(0, 20, by = 0.5)
logistic_growth <- tibble(
  t = t_seq,
  y = 100 / (1 + exp(-0.5 * (t - 10))) + rnorm(length(t_seq), sd = 3)
)

# Fit with nls() (non-linear least squares)
mod_nls <- nls(y ~ K / (1 + exp(-r * (t - t0))),
  data      = logistic_growth,
  start     = list(K = 100, r = 0.5, t0 = 10))

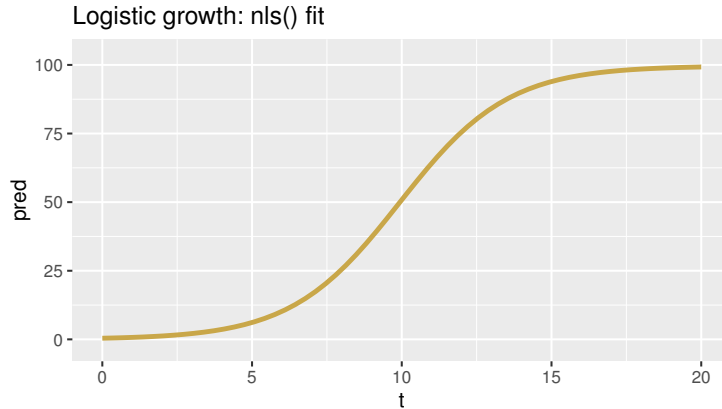
coef(mod_nls)
##           K           r           t0
```

Case Study 3: Non-Linear Growth Models II

```
##    99.73      0.496      9.98    <- recovered true parameters

# Predictions
logistic_growth |>
  data_grid(t = seq_range(t, 60)) |>
  add_predictions(mod_nls) |>
  ggplot(aes(t, pred)) +
  geom_line(colour = "#C9A84C", linewidth = 1.2) +
  geom_point(data = logistic_growth, aes(y = y),
             colour = "#1B3A6B", alpha = 0.6) +
  labs(title = "Logistic growth: nls() fit")
```

Case Study 3: Non-Linear Growth Models III



► Interpretation

`nls()` fits **non-linear** models by iterative least squares. It requires good starting values (use domain knowledge or a grid search). The recovered parameters ($K = 99.7$, $r = 0.50$, $t_0 = 9.98$) are very close to the true generating values.

8

Summary and Exercises

1 — Define

Every model needs: (a) a family (the equation shape), and (b) a fitting criterion (usually RMSE for OLS).

3 — Visualise

Always plot (a) the fitted model against the data and (b) residuals vs predictors, vs fitted values, and the QQ-plot.

2 — Fit

Use `lm()` for all standard linear models. Use `optim()` only when the loss function is non-standard.

4 — Extend

Use `*` for interactions, `I()` to protect arithmetic, `ns()` for smooth non-linear terms.

5 — Scale Up

`purrr::map() + nest() + broom::tidy()`
let you fit, summarise, and compare hundreds of models at once.

6 — Choose Wisely

Match the model family to the response type:
OLS for continuous, GLM for counts/binary,
GAM for flexible curves, robust models for
contaminated data.

- E1.** Fit a linear model to `sim1a` (Student- t noise, $df = 2$). Rerun 10 times with different seeds. Compare OLS and MAE-based fitting via `optim()`. Which is more stable under outliers?
- E2.** Use `model_matrix()` to inspect $y \sim x1 * x2$ for `sim3`. Write out the full model equation using the estimated coefficients.
- E3.** Fit `loess(y ~ x)` to `sim1`. Compare predictions and residuals to those from `lm()`. Is the curve similar to what `geom_smooth()` produces?
- E4.** Fit natural splines `ns(x, df)` for $df = 1, 2, 3, 4, 5$ to `sim5`. Use 10-fold CV to choose the optimal df .
- E5.** Using `sim4`, demonstrate via residual plots and AIC that the interaction model $(x1 * x2)$ fits better than the additive model $(x1 + x2)$.

- E6. Apply the diamonds log-log model to the first 500 rows. Use `broom::augment()` to identify the five diamonds with the largest absolute residuals. What do they have in common?
- E7. Replicate the `nycflights13` many-models analysis using `origin` as the grouping variable instead of `carrier`.



Wickham, H., & Grolemund, G. (2017).

R for Data Science.

O'Reilly Media.

Available at: <https://r4ds.had.co.nz/model-basics.html>



Wickham, H. (2016).

ggplot2: Elegant Graphics for Data Analysis (2nd ed.).

Springer.



Wood, S. N. (2017).

Generalised Additive Models: An Introduction with R (2nd ed.).

CRC Press.



Friedman, J., Hastie, T., & Tibshirani, R. (2010).

Regularisation Paths for Generalised Linear Models via Coordinate Descent.

Journal of Statistical Software, 33(1), 1–22.



Venables, W. N., & Ripley, B. D. (2002).
Modern Applied Statistics with S (4th ed.).
Springer.



R Core Team. (2024).
R: A Language and Environment for Statistical Computing.
R Foundation for Statistical Computing, Vienna, Austria.